System Analysis Documentation

Project Title: HireFlow – Decision Engine for Recruitment Workflows

Domain: Domain 1 – AI Systems & Agentic Workflow Automation

Group Name: Fantastic Four

Team member 1	Tsang Da Xin
Team member 2	Patrick Woo Soon Tat
Team member 3	Tan E Jane
Team member 4	Nareen Raj

Contents

1.0 Introduction 4
1.1 Purpose 4
1.2 Background 5
1.3 Target Stakeholders 6
2.0 System Architecture & Design 7
2.1 High-Level Architecture 7
2.1.1 System Overview 7
2.1.2 Architecture Type 9
2.1.3 Dependency Diagram 10
2.1.4 Sequence Diagram 12
2.2 Technology Stack 13
2.3 Key Data Flows 13
2.3.1 Data Flow Diagram (DFD) 13
2.3.2 Normalized Database Schema 15
3.0 Functional Requirements & Scope 16
3.1 Minimum Viable Product (MVP) 16
3.2 Non-Functional Requirements (NFRs) 16
3.3 Out of Scope / Future Enhancements 18
3.4 Monitoring, Evaluation, Assumptions & Dependencies 18
3.4.1 Technical Evaluation 18
3.4.1.1 Grayscale Rollout & Controlled Evaluation 18
3.4.1.2 Emergency Rollback (ER) & Safe Recovery 19
3.4.2 Priority Matrix 19
3.4.3 Monitoring Plan 20
3.4.4 Assumptions 20
3.4.5 External Dependencies 20

4.0 Project Management & Team Contributions 22

4.1 Project Timeline 22

4.2 Team Roles & Responsibilities 23

4.3 Recommendations 23

1.0 Introduction

HireFlow is an AI-powered recruitment decision system designed to address inefficiencies in high-volume hiring workflows. It introduces a stateful, agentic architecture that transforms traditional recruitment processes that typically reliant on manual coordination and fragmented tools into a structured, automated, and decision-driven workflow.

The system leverages Z AI GLM as a core reasoning engine to process unstructured inputs such as resumes, job descriptions, and interview feedback. These inputs are converted into structured representations that enable consistent candidate evaluation, decision routing, and workflow execution across multiple hiring stages.

Unlike conventional Applicant Tracking Systems (ATS) that function primarily as data repositories, HireFlow operates as an **active decision engine**, capable of both generating insights and executing workflow actions such as candidate routing, scheduling, and recommendation generation.

1.1 Purpose

This System Analysis Document (SAD) defines the technical architecture, system design, and engineering decisions underlying the HireFlow platform.

The document outlines the key technical elements of the system, including:

1. **Architecture**
   The overall system structure, including frontend, backend services, workflow orchestration layer, database, and integration with external APIs and the Z AI GLM reasoning service.
2. **Data Flows**
   The movement of data across the system, including how unstructured inputs such as resumes and job descriptions are processed, transformed into structured representations, and propagated through decision-making workflows.
3. **Model Process**
   The role of Z.AI GLM as a reasoning service layer, including how prompts are constructed, how context is managed within token limits, how responses are parsed into structured outputs, and how these outputs drive downstream workflow decisions.

4. **Role of Reference**
   This document serves as a technical reference for developers, system designers, and evaluators to understand the system's architecture, workflow logic, and integration design, ensuring consistency in implementation and evaluation.

1.2 Background

Modern recruitment workflows in mid-sized organizations remain heavily dependent on manual coordination, despite the availability of digital tools such as Applicant Tracking Systems (ATS). Recruiters frequently switch between resumes, emails, calendars, and interview feedback systems, resulting in fragmented workflows and inconsistent decision-making.

While early AI solutions introduced automation into recruitment, they are typically limited to isolated tasks such as resume screening or text summarization. These systems lack the ability to maintain **decision context across multiple stages**, leading to disconnected outputs and reduced reliability in hiring outcomes.

Furthermore, increasing regulatory expectations such as the Personal Data Protection Act (PDPA) 2026 require organizations to ensure **transparency, traceability, and consistency** in automated decision-making processes. Traditional systems are not designed to meet these requirements at scale.

HireFlow addresses these limitations by introducing a **stateful, workflow-driven architecture** that enables:

- Structured representation of both candidate data and job requirements
- Multi-step reasoning across hiring stages
- Consistent and explainable decision outputs
- Automated execution of workflow actions with human oversight

This positions HireFlow as a system that moves beyond static automation toward **continuous, context-aware decision orchestration** in recruitment workflows.

1.3 Target Stakeholders

Stakeholders	Roles	Expectations
TA Manager / HR Lead (Primary User)	Oversees hiring pipelines, reviews AI-generated candidate recommendations, resolves decision conflicts, and approves or overrides hiring decisions	Clear visibility of candidate pipeline, reduced manual workload, consistent and explainable decision support, and improved hiring efficiency
Hiring Manager	Reviews structured candidate evaluations, provides interview feedback, and contributes to final hiring decisions	Structured and explainable candidate insights, consistent evaluation criteria, and reduced reliance on subjective judgment
Candidate (Secondary Actor)	Submits application materials, receives updates, and interacts with scheduling workflows	Transparent application status, timely updates, and flexible interview scheduling experience
Development Team	Designs, implements, and maintains system architecture, APIs, and GLM integration	Well-defined architecture, modular codebase, clear data flow, and stable integration with external services such as GLM API, calendar APIs
QA Team	Validates system behaviour across workflows and ensures reliability of AI-assisted decision processes	Clearly defined system scope, structured input/output formats, testable workflows, and coverage of edge cases and failure scenarios

2.0 System Architecture & Design

2.1 High-Level Architecture

2.1.1 System Overview

Type	Details
System	Web-based AI Recruitment Decision Platform (Workflow Dashboard + Candidate Interaction Layer)
Architecture	Client–server architecture with modular backend services, stateful workflow orchestration, and Z AI GLM integrated as a reasoning service layer

The HireFlow **frontend** acts as the **visual interaction layer** for the Workflow Orchestrator, translating complex, stateful GLM decisions into an intuitive dashboard for HR teams. The interface is compartmentalized into five core modules:

- **System Overview Dashboard:** The central command center displaying high-level recruitment metrics, pipeline visualizations, talent trends, and real-time AI orchestrator activity.
- **Resume Screening Interface:** An automated triage board where uploaded resumes are parsed by the GLM agent and autonomously routed into distinct Kanban columns (Screening, HR Intervention, Rejected), complete with explainable AI reasoning and manual override controls.
- **Interview Hub:** A post-interview database featuring session transcripts, an interactive AI Copilot for deep-dive Q&A, and a Hybrid Decision Engine that merges AI and human scores to automatically trigger the final hiring verdict (auto-sending offer letters).
- **Employee Onboarding Dashboard:** A dynamic task tracker that monitors pending employee and internal IT requirements, utilizing the AI agent to autonomously send follow-up reminders and eliminate bottlenecks.
- **Orchestrator Logs:** A technical observability dashboard providing a live feed of system activities, API calls, and raw AI payloads for complete transparency and auditing.

These clients communicate with the backend API layer, which manages request handling, workflow state transitions, structured data persistence, and integration with external services.

At the core of HireFlow is a Workflow Orchestrator, which maintains candidate state across all hiring stages and coordinates decision execution. Unlike traditional systems that perform isolated API calls, this orchestrator manages stateful, multi-step workflows, where each stage (profile extraction, matching, decision, scheduling) is executed sequentially based on structured outputs generated by the Z.AI GLM. This enables HireFlow to function as a continuous decision-making system, rather than a collection of disconnected automation tools.

The backend, built using **Python (FastAPI)**, is structured into the following key components:

- **Application API Layer:** Handles communication between the frontend (React) and backend services, exposing endpoints for user interaction and workflow triggers.
- **Workflow Orchestrator:** Controls stage transitions, manages workflow state, and determines the next action such as shortlist, schedule or notify.
- **Prompt Builder / Context Manager:** Constructs structured prompts by combining candidate profiles, job criteria, and workflow context before sending requests to the GLM.
- **Response Parser / Validator:** Processes GLM outputs and converts them into structured, validated JSON formats for downstream execution.
- **Tool Integration Layer:** Integrates with external services such as **Google Calendar API** and **Gmail API** for scheduling and notification automation.
- **State Store / Database Layer:** Uses **SQLite** to persist candidate profiles, workflow stages, decisions, logs, and system state.

This architecture allows HireFlow to operate not as a static Applicant Tracking System (ATS), but as a **stateful decision engine** that:

- Maintains continuity across hiring stages
- Executes multi-step workflows autonomously
- Ensures decisions are structured, traceable, and explainable

2.1.2 Architecture Type

In HireFlow, Z AI GLM is integrated as a **reasoning service layer**, not as a generic AI component. It is invoked through controlled backend services that construct prompts, manage context, enforce token limits, and validate structured responses before workflow execution continues.

The GLM service layer is responsible for supporting several reasoning tasks across the hiring workflow, including:

- **Candidate Profile Extraction** from unstructured resumes
- **Structured Job Criteria Interpretation** from job descriptions
- **Candidate–Role Matching and Recommendation Generation**
- **Interview Feedback Structuring**
- **Decision Support for workflow routing and hiring recommendations**

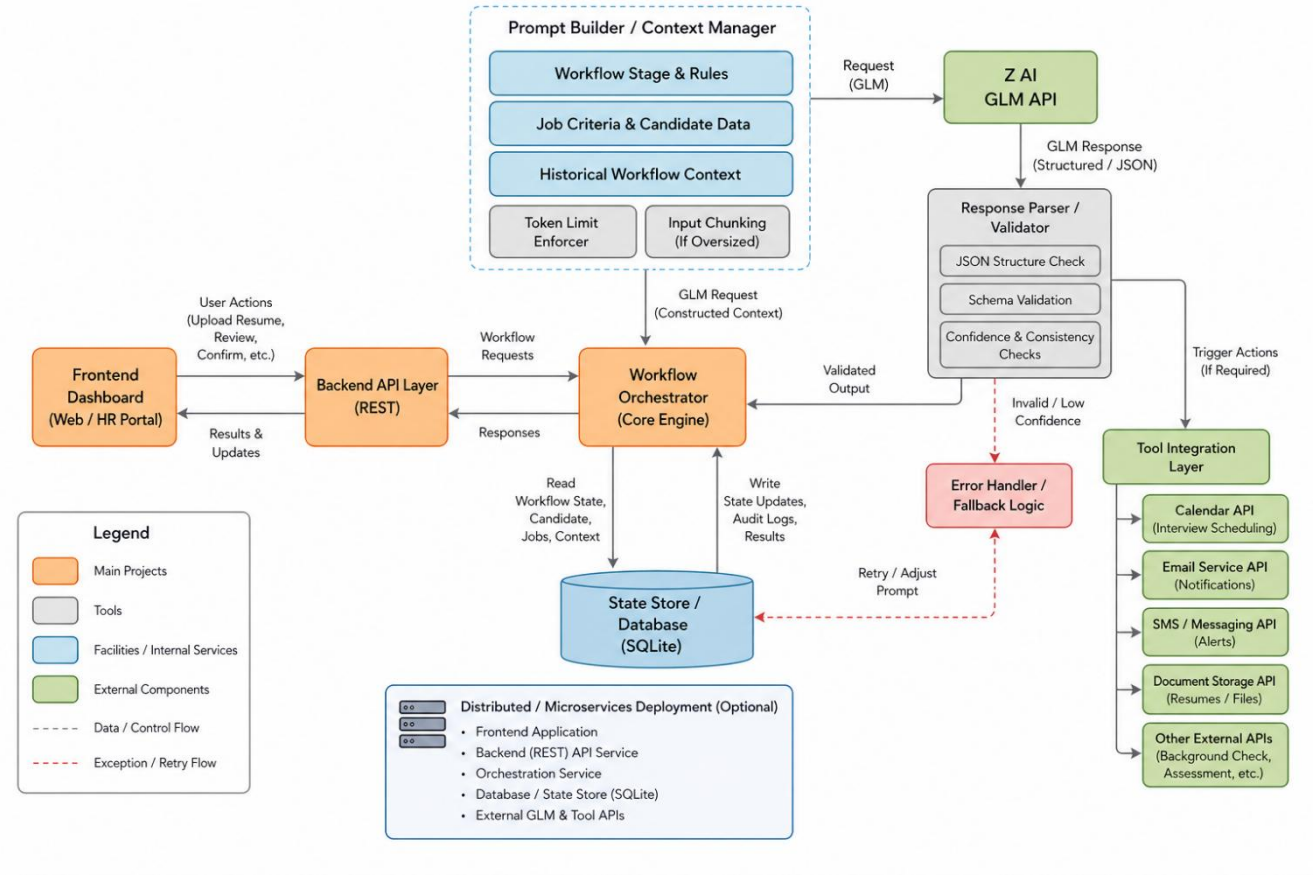
The interaction with GLM follows a controlled pipeline:

1. The **Workflow Orchestrator** determines the current workflow stage and required reasoning task
2. The **Prompt Builder** assembles task instructions, structured context, and relevant candidate/job data
3. The **Context Manager** enforces input size limits, applies chunking when needed, and filters irrelevant content before submission
4. The request is sent to the **Z AI GLM API**
5. The response is received by the backend and passed through a **Parser / Validator**
6. Structured outputs are stored in the database and forwarded to the next system component, such as the matching engine, routing engine, or scheduling layer

This design ensures that GLM outputs are treated as **controlled service responses** rather than free-form text, allowing them to drive downstream workflow logic in a traceable and machine-readable manner.

This design ensures that GLM outputs are treated as **controlled, machine-readable service responses**, rather than free-form text.

2.1.3 Dependency Diagram



The dependency structure of HireFlow illustrates how frontend, backend, orchestration logic, GLM reasoning services, external tool APIs, and persistent storage interact to execute recruitment workflows.

The major dependency flow is as follows:

- The **Frontend Dashboard** sends user actions such as upload resume, review recommendation, confirm decision to the **Backend API Layer**
- The **Backend API Layer** forwards workflow-related requests to the **Workflow Orchestrator**
- The **Workflow Orchestrator** retrieves relevant data from the **State Store or Database** and determines the required reasoning task
- The **Prompt Builder (Context Manager)** constructs the GLM request using:
  - current workflow stage
  - structured job criteria
  - candidate data
  - relevant historical workflow context

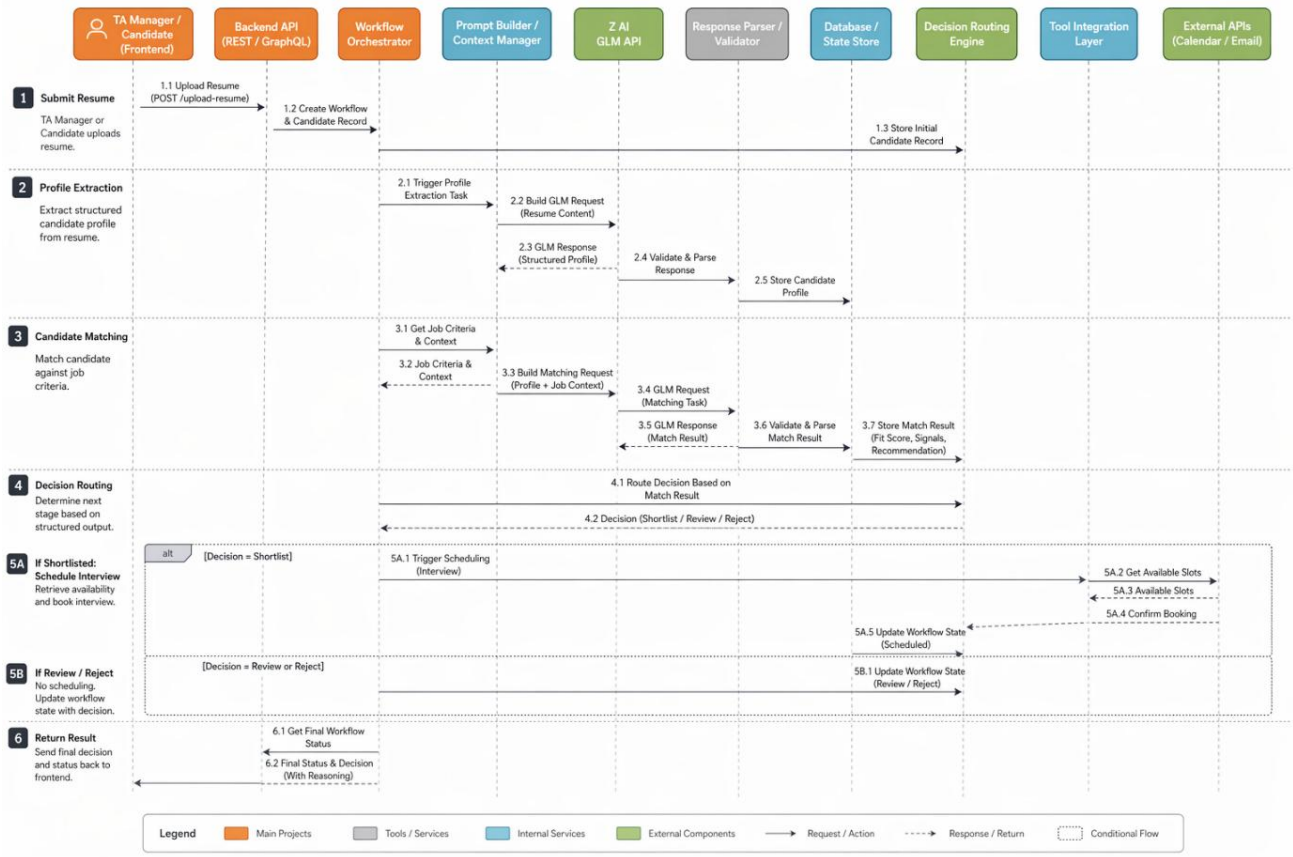
- Before dispatch, the **Context Manager** enforces token limits and segments oversized inputs through chunking
- The request is sent to the **Z AI GLM API**
- The returned response is received by the **Response Parser or Validator**, which:
  - checks JSON structure
  - validates schema compliance
  - flags inconsistent or low-confidence outputs
- Valid structured outputs are then passed back to the **Workflow Orchestrator**
- The orchestrator updates workflow state in the **Database** and, where required, triggers the **Tool Integration Layer** for actions such as interview scheduling via calendar or email APIs
- Final results are returned to the **Frontend** for display to HR users

If deployed in a distributed setup, the architecture can separate:

- frontend application
- backend API service
- orchestration service
- database
- external GLM and tool APIs

This separation improves modularity, maintainability, and service isolation.

2.1.4 Sequence Diagram



The sequence diagram for HireFlow illustrates the end-to-end execution of the **early-stage hiring decision workflow**, which is the primary MVP scenario demonstrated in the hackathon prototype.

The sequence begins when a **TA Manager uploads a resume** through the frontend interface. The request is submitted to the backend API, which creates a workflow instance and stores the initial candidate record.

Next, the **Workflow Orchestrator** triggers the profile extraction task. Relevant resume content is prepared by the prompt builder and sent to Z AI GLM, which returns a structured candidate profile. The response is validated and stored in the database.

The orchestrator then retrieves the relevant job criteria and initiates the candidate matching task. A new GLM request is constructed using the structured candidate profile and job context. GLM returns a match result containing fit scores, reasoning signals, and a recommendation.

Based on the structured output, the **Decision Routing Engine** determines the next stage:

- shortlist
- review
- reject

If the candidate is shortlisted, the orchestrator triggers the scheduling flow through the **Tool Integration Layer**, which connects to external calendar or email APIs. Available time slots are retrieved, a booking is confirmed, and the workflow state is updated.

Finally, the updated decision and scheduling status are returned to the frontend, where the TA Manager can review the result and the candidate can receive status updates.

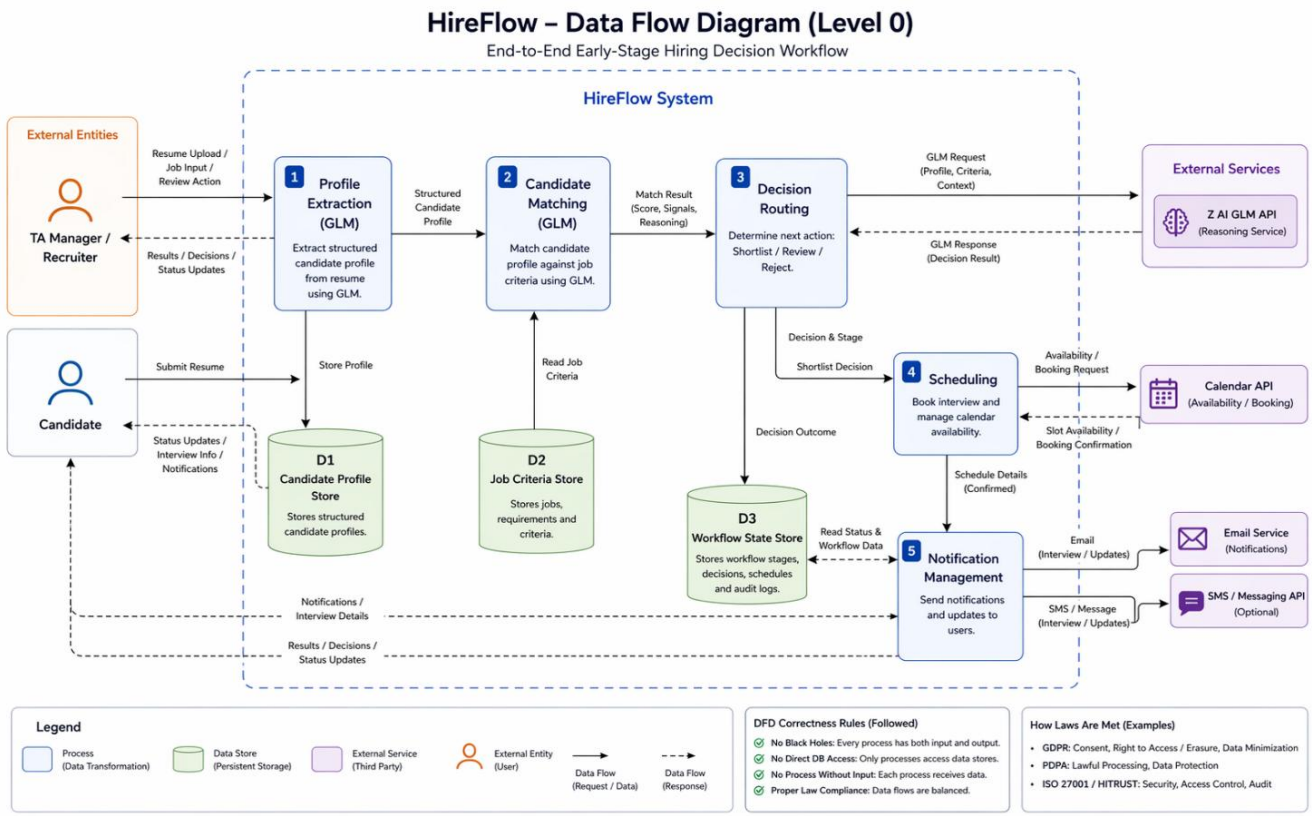
This sequence demonstrates how HireFlow integrates structured reasoning, workflow state management, and external tool execution into one continuous recruitment process.

2.2 Technology Stack

1. Frontend: React (TypeScript), Express.js
2. Backend: FastAPI (Python)
3. Database: SQLite

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD)



External Entities

- TA Manager / Recruiter
- Candidate

Processes

1. Profile Extraction (GLM)
2. Candidate Matching (GLM)
3. Decision Routing
4. Scheduling
5. Notification Management

Data Stores

- D1 Candidate Profile Store
- D2 Job Criteria Store
- D3 Workflow State Store

External Services

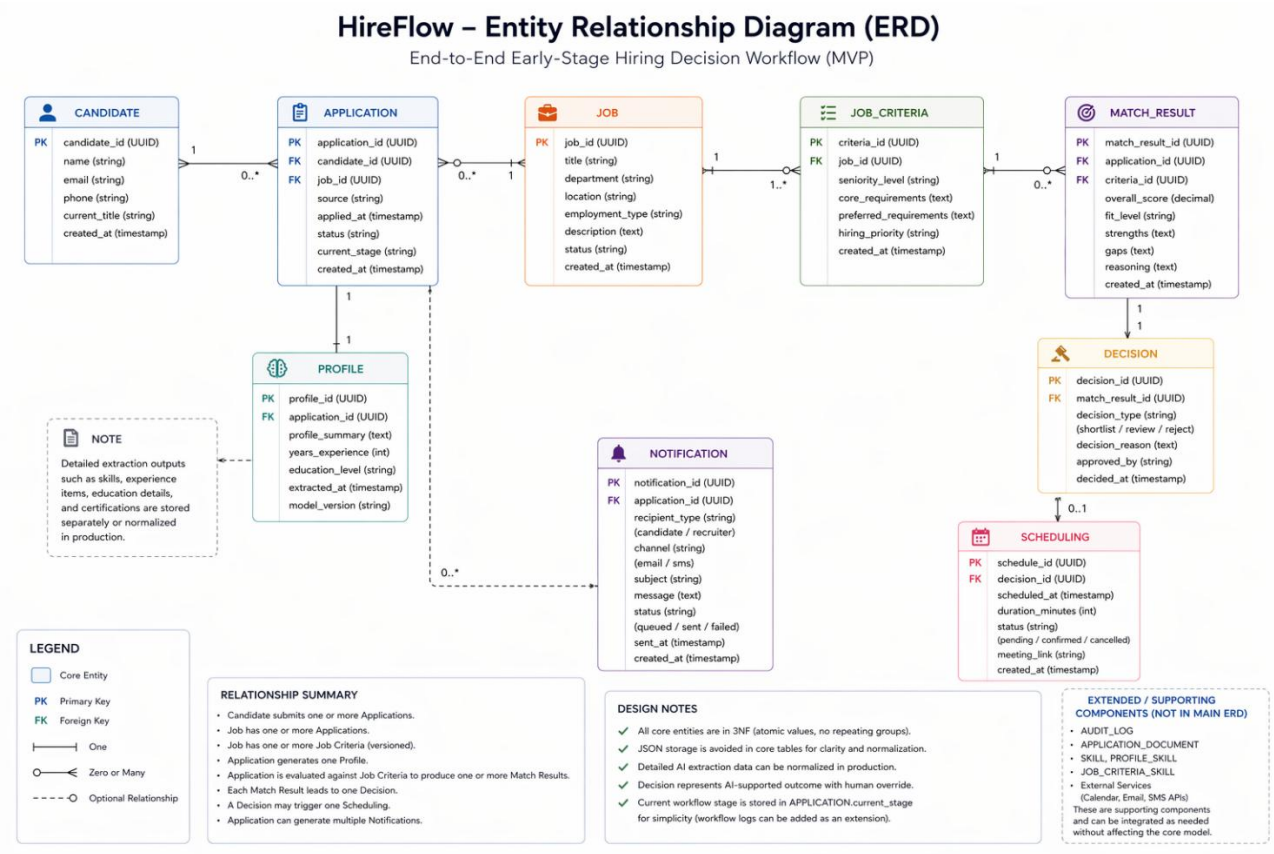
- Z AI GLM API
- Calendar API
- Email Service
- SMS / Messaging API (Optional)

Data Movement

- Resume upload, job input, and review actions are submitted by the TA Manager / Recruiter to the system.
- The system performs Profile Extraction (GLM) to convert resume input into a structured candidate profile, which is stored in D1 Candidate Profile Store.
- The Candidate Matching (GLM) process reads candidate profile data from D1 and job requirements from D2 Job Criteria Store to generate a match result containing score, signals, and reasoning.
- The Decision Routing process uses the match result and sends a GLM request with profile, criteria, and context to the Z AI GLM API, then receives a structured decision response.
- The resulting decision and workflow stage are stored in D3 Workflow State Store.
- If the candidate is shortlisted, the Scheduling process interacts with the Calendar API to retrieve availability and confirm interview booking.
- The Notification Management process reads workflow state and schedule details, then sends updates through the Email Service and optional SMS / Messaging API.

- Final results, decisions, status updates, and interview notifications are returned to the TA Manager / Recruiter and Candidate.

2.3.2 Normalized Database Schema



The ERD models the MVP recruitment workflow in a normalized relational structure. The core design centres on candidate applications, structured profile extraction, job criteria evaluation, match results, decision routing, interview scheduling, and notification delivery. The schema resolves key relationships through the APPLICATION entity and keeps workflow complexity focused on the early-stage hiring decision loop.

3.0 Functional Requirements & Scope

This section defines the **core functional boundaries** of HireFlow and ensures that the project remains focused on a **high-impact MVP** that can be realistically built and demonstrated during UMHackathon 2026. The scope emphasizes the early-stage hiring decision loop, where structured AI reasoning can deliver the greatest operational impact.

3.1 Minimum Viable Product (MVP)

The following are the **core features** to be implemented and demonstrated by the end of UMHackathon 2026:

No	Feature	Description
1	**Resume Submission & Profile Extraction**	TA Manager uploads a resume, and the system uses Z AI GLM to extract a structured candidate profile from unstructured resume content.
2	**Job Criteria Modeling & Candidate Matching**	The system converts job requirements into structured criteria and evaluates candidate-job alignment using GLM-powered matching, producing fit scores, strengths, gaps, and reasoning.
3	**AI-Driven Decision Routing**	Based on the structured match result, the system determines the next workflow stage such as shortlist, hr intervention, or reject, and updates the workflow state accordingly.
4	**Interview Scheduling Trigger**	If a candidate is shortlisted, the system triggers scheduling by retrieving available slots through calendar integration and confirming interview details.
5	**Notification & Status Updates**	The system sends candidate and recruiter updates regarding application status, decisions, and scheduling outcomes through supported communication channels.

3.2 Non-Functional Requirements (NFRs)

The following non-functional requirements define the key system qualities required to ensure that HireFlow is technically feasible, reliable, and maintainable.

Quality	Requirement	Implementation
Scalability	The system should support multiple concurrent candidate workflows without corrupting workflow state or causing inconsistent decisions.	Use modular backend services with separated orchestration, reasoning, and persistence responsibilities. Workflow state is stored persistently to support multiple active sessions.
Reliability	Candidate workflow states, match results, and decisions must remain consistent even if external services fail.	Persist workflow state in the database, validate structured outputs before use, and apply retry/fallback logic for GLM and external API failures.
Maintainability	The system should be easy to extend and debug as new hiring stages or integrations are introduced.	Separate components into frontend, backend API, workflow orchestrator, GLM service layer, parser/validator, and tool integration layer. Use clear API contracts and modular service responsibilities.
Token Latency	GLM-assisted reasoning tasks should return within an acceptable response time under normal load.	Use prompt scoping, context filtering, token limits, and chunking for oversized inputs. Display loading states in the frontend while awaiting structured responses.
Cost Efficiency	Average reasoning cost per workflow session must remain within a practical budget range for MVP usage.	Reuse structured outputs, cache job criteria representations, limit multi-step reasoning to high-impact stages, and avoid repeated full-context prompting.
Security & Privacy	Candidate resumes and workflow data must be handled securely and only accessed by authorized components.	Enforce role-based access control, secure transport for API calls, protected database access, and storage of only necessary candidate data in line with data minimization principles.
Explainability	AI-assisted outputs should be understandable and reviewable by HR users.	Store structured reasoning summaries, match explanations, and decision

		signals so they can be surfaced in the dashboard for human review.

3.3 Out of Scope / Future Enhancements

The following features are part of the broader product vision but are **not included in the UMHackathon 2026 MVP**:

a) Real-time conversational chat between candidates and recruiters
b) Advanced analytics dashboards for long-term hiring performance tracking
c) Full enterprise ATS migration and legacy HR system integration
d) Deep normalization of all extracted AI profile attributes into production-grade relational skill and experience tables
e) Multi-round interview orchestration and post-hire employee lifecycle management

3.4 Monitoring, Evaluation, Assumptions & Dependencies

3.4.1 Technical Evaluation

3.4.1.1 Grayscale Rollout & Controlled Evaluation

For the hackathon prototype, HireFlow will be evaluated in a controlled demo environment rather than a full production rollout. The team will simulate staged usage by testing the system across selected workflow scenarios:

- valid resume → strong scoring → shortlist → scheduling
- valid resume → weak scoring → review/reject
- incomplete or low-quality input → validation / fallback path

Where appropriate, lightweight comparison can be made between:

- structured AI-assisted routing vs manual rule-only routing
- different prompt variants for profile extraction or match explanation

This allows the team to observe system behaviour, validate workflow consistency, and assess whether the GLM-assisted pipeline improves decision clarity and efficiency.

3.4.1.2 Emergency Rollback (ER) & Safe Recovery

If critical issues occur during evaluation or deployment, HireFlow will apply a safe recovery approach:

- If GLM responses fail validation, the workflow does not proceed automatically and is routed to fallback handling or manual review
- If calendar or notification APIs fail, scheduling and communication actions are retried or queued without corrupting workflow state
- If a deployment issue affects core workflow execution, the team reverts to the last stable application version and uses preserved database state to restore continuity

For MVP, the rollback strategy focuses on workflow safety and state recovery, rather than full production-grade deployment orchestration.

3.4.2 Priority Matrix

Priority	Trigger Condition	Action
P1 – Critical	GLM output is invalid for core profile extraction or decision routing, preventing workflow continuation	Stop automated progression, trigger fallback validation path, and route for manual review
P1 – Critical	Workflow state becomes inconsistent or cannot be written/read correctly	Freeze affected workflow, restore from last valid persisted state, and block downstream actions
P2 – High	Calendar API or notification service fails during scheduling or updates	Retry request, queue the failed action, and notify user of temporary delay
P2 – High	Response latency becomes too high for core reasoning tasks	Reduce context size, enforce chunking, and use simplified prompt path if needed
P3 – Medium	Non-critical UI or reporting issue occurs	Log issue, continue core workflow, and patch in next iteration

3.4.3 Monitoring Plan

The system will monitor the following operational signals during testing and demo execution:

- GLM request success/failure rate
- JSON validation failure rate
- workflow transition success rate
- scheduling success/failure rate
- notification delivery success/failure rate
- average reasoning latency per workflow stage
- fallback / retry invocation count

Monitoring outputs can be stored in logs and summarized through simple internal reporting to help the team detect failures, inconsistencies, or performance bottlenecks during evaluation.

3.4.4 Assumptions

The development and deployment of HireFlow assume the following conditions:

- Candidates and recruiters provide resumes and job descriptions in text-based or machine-readable PDF formats
- Internet connectivity is available for frontend access, GLM API calls, and third-party API integrations
- Calendar and notification APIs are accessible during demonstration and testing
- Human users remain available to review or approve decisions where fallback or conflict handling is required
- The MVP focuses on early-stage hiring workflows and does not need full enterprise HR process coverage

3.4.5 External Dependencies

Tool / Service	Purpose	Risk
Z AI GLM API	Core reasoning engine for profile extraction, candidate matching, and decision support	High – if unavailable or rate-limited, core workflow intelligence is disrupted

Calendar API	Retrieves slot availability and supports interview scheduling	Medium – if unavailable, scheduling flow is delayed or falls back to manual handling
Email / Notification Service	Sends application updates, interview confirmations, and status messages	Medium – delivery failures may affect user communication but not core workflow state
File Storage Service	Stores uploaded resumes and related documents	Medium – file upload/retrieval issues may affect profile extraction inputs
Database / State Store	Persists candidates, applications, decisions, and workflow state	High – failure impacts continuity, consistency, and auditability of workflow execution

4.0 Project Management & Team Contributions

The development of HireFlow is structured as a focused 10-day sprint, prioritizing rapid prototyping of a high-impact MVP while ensuring technical feasibility, system coherence, and demonstration readiness.

4.1 Project Timeline

The project is executed in four structured phases, balancing product thinking, system design, implementation, and demo readiness.

Phase	Timeline	Focus	Justification
Phase 1: Problem Framing & System Design	Day 1 – 2	Define problem scope, target users, MVP features, system architecture, and data model (ERD, DFD, sequence diagram)	Early clarity reduces rework risk and ensures all components align with judging criteria (product thinking + architecture)
Phase 2: Core System Development	Day 3 – 6	Implement backend API, workflow orchestration logic, GLM integration (profile extraction + matching), and database schema	Focus on building the **core decision loop (profile → match → decision)** first to ensure MVP completeness
Phase 3: Integration & Feature Completion	Day 7 – 8	Integrate scheduling (calendar API), notification system, and frontend dashboard for interaction and visualization	Ensures end-to-end workflow demonstration from input to actionable output
Phase 4: Testing, Optimization & Demo Preparation	Day 9 – 10	Conduct system testing, handle edge cases, optimize prompts and latency, and prepare demo narrative and presentation	Critical for ensuring system stability, clarity of demo story, and alignment with judging expectations

4.2 Team Roles & Responsibilities

The team is structured to reflect **real-world system development roles**, ensuring clear ownership, parallel progress, and accountability.

Team Member	Role	Responsibility
Patrick Woo Soon Tat	Backend & Orchestration Lead	Designs and implements backend API layer and workflow orchestrator. Handles state management, decision routing logic, and integration across system components.
Nareen Raj	AI / GLM Integration Lead	Designs prompt strategy, context construction, and structured output validation. Implements profile extraction, candidate matching, and reasoning pipeline using Z AI GLM.
Tsang Da Xin	Frontend & UX Lead	Develops recruiter dashboard interface. Implements candidate pipeline visualization, decision outputs, and interaction flows to support a clear and intuitive demo experience.
Tan E Jane	Data & Integration Lead	Designs ERD, DFD, and sequence diagrams. Ensures schema consistency and data integrity, and handles external API integration (calendar, notifications), testing workflows, and system reliability.

4.3 Recommendations

To improve scalability, performance, and reliability beyond the MVP, the following enhancements are recommended:

1. **Caching & Cost Optimization**

   To reduce operational cost and improve performance, caching mechanisms such as Redis can be introduced for frequently reused data, including job criteria representations and structured candidate profiles. By avoiding repeated GLM calls for the same inputs and reusing previously computed outputs, the system can significantly lower token usage while improving response time and overall efficiency.

2. **Asynchronous Workflow Execution**

   Workflow steps such as candidate matching, scheduling, and notification can be executed asynchronously using job queues. This prevents blocking of the main application flow during long-running operations, especially when interacting with external APIs, and improves system responsiveness and user experience.

3. **Scalable Service Architecture**

   The system can be extended into a modular, service-oriented architecture by separating components such as frontend, backend API, workflow orchestration, database, and external integrations into independently deployable services. This enhances scalability, maintainability, and allows the system to evolve into a microservices-based architecture in production.

4. **Enhanced Data Normalization**

   AI-extracted attributes such as skills, experience, and education can be further normalized into structured relational tables rather than simplified text fields. This improves data consistency, reduces redundancy, and enables more advanced querying, filtering, and analytical capabilities for long-term system use.

5. **Monitoring & Observability**

   Implementing logging and monitoring mechanisms allows the system to track key metrics such as GLM response quality, workflow transition success rates, and external API failures. This improves debugging capabilities, enables early detection of system issues, and supports continuous improvement of reliability and performance.

6. **Robust Fallback Mechanism**

   To ensure system reliability, fallback mechanisms can be enhanced by incorporating rule-based decision support when GLM outputs fail or are unreliable, along with a manual override interface for recruiters. This ensures workflow continuity even under failure conditions and increases user trust in AI-assisted decision-making.